

Augur — Project Overview

OSS Dependency Risk Intelligence Platform · Version 2

Author: Alae Rhazouani · **Profile target:** AI Systems Engineer · **Duration:** 12 weeks (solo)

1. Overview

Augur is a system that ranks open-source packages on PyPI and npm by the risk of a vulnerability advisory being published against them in the next 90 days, and then measures honestly whether that ranking is any better than ranking by popularity. It monitors the public health signals of the top 10,000 packages — contributor activity, release cadence, maintainer changes, dependency graphs, and OpenSSF Scorecard checks — and learns which of those signals, if any, carry information about near-term risk.

A separate module watches new releases for the behavioural fingerprints of a maintainer takeover. On top of both sits a conversational interface: a security engineer can ask whether a dependency tree is safe to ship and get an answer in which every factual claim is tied to an advisory record the system actually retrieved. The whole thing re-scores itself weekly against newly published advisories, and the CI pipeline blocks changes that make it worse.

The project exists to demonstrate one core engineering thesis: **that an AI system can be useful in production only if it is observable, evaluable, and operationally maintained — not just accurate on a demo day.** Version 2 adds a second thesis, which is about the science rather than the engineering: that a prediction claim is worth nothing until it has been measured against a baseline and cleared of leakage.

The problem it solves

Today's dependency security tools (Dependabot, Snyk, Renovate) are reactive. They match your dependencies against a database of *already-published* advisories, which means you only find out about a risk after it has been publicly disclosed. The commercial tools that attempt to look forward (Socket.dev, Snyk) are closed and expensive, and none of them publish their baselines or their error rates.

Augur reframes the problem as triage rather than prophecy. Given finite review time, which dependencies deserve attention first? That is a ranking problem, and ranking problems can be evaluated rigorously against a held-out future. "We predict CVEs before they happen" cannot be evaluated rigorously, because the claim is not precise enough to fail.

What changed from version 1, and why

The first version of this plan had a compelling story and a weak core. Nine specific things were wrong with it, and each has a concrete fix in version 2.

Gap in version 1	Fix in version 2
One model for "any CVE" plus takeover signals	Two separate tracks with separate labels, methods and metrics — ordinary bugs and malicious takeovers have different causes
No baseline at all	A three-rung baseline ladder; lift over popularity is the headline number
Labels anchored on CVE publication date	Anchored on the earliest disclosure signal, with a seven-day blackout window and strict time-based splits
Pre-written metrics (0.78 / 0.71 / 0.89)	No target numbers anywhere; every figure comes from the test set with a confidence interval, after the build
XZ Utils as the headline demo	Design inspiration only; evaluation runs on held-out npm and PyPI incidents, led by event-stream

Gap in version 1	Fix in version 2
Temporal Fusion Transformer as the model	LightGBM on windowed tabular features; TFT retained only as a stretch goal that must beat it
Live GitHub API for historical reconstruction	BigQuery public datasets (GH Archive, Scorecard, deps.dev) for history; APIs only for the daily refresh
"Five agents"	Honest component names; only the assistant is an agent
Thirteen technologies, seven of them new	Core first, Kubernetes and Airflow last, with explicit kill criteria at weeks 4, 8 and 12

Why this domain was chosen over alternatives

The project was originally scoped around environmental sensor data, which works technically but is an oversaturated domain that does not differentiate you in an interview. Supply chain security was chosen instead for three concrete reasons. The ground truth is dated and public: an advisory either appears for a package within the prediction window or it does not. The historical dataset already exists and is free, through OSV, GH Archive and the OpenSSF Scorecard, all queryable from BigQuery. And the problem is instantly legible to any technical interviewer, because every software company has a dependency tree and has felt the pain of a surprise vulnerability disclosure.

What version 1 got wrong was assuming that clean ground truth automatically produces a good predictor. It does not. Most advisories describe ordinary bugs found by researchers and fuzzers, and those correlate with popularity, code size and scrutiny — not with the contributor exodus and suspicious-maintainer signals the original design was built around. Popular, actively maintained packages get *more* advisories precisely because more people audit them, so a naive model learns "busy and popular equals risky", which is the opposite of the intended narrative. Version 2 confronts that confounding head-on instead of hoping it will not appear.

2. System architecture

Augur is built as five components coordinated by a custom Python orchestrator — deliberately not built on LangChain or LangGraph, because the goal is to demonstrate that you understand coordination at the level of shared state, routing logic and failure handling, not that you can configure a third-party framework.

Version 1 called all five of these "agents". Two of them were an ETL job and an evaluation pipeline, and senior engineers discount projects that label cron jobs as agents. Naming them honestly reads as more mature, not less ambitious.

Component 1 — Ingestion pipeline

Historical data comes from BigQuery public datasets, not the live GitHub API: GH Archive for commits, contributors, issues and releases; the OpenSSF Scorecard dataset for security-practice checks; PyPI download statistics; and deps.dev for package-to-repository mapping and dependency graphs. Labels come from the OSV.dev bulk export, which is ecosystem-native for PyPI and npm and avoids the messy business of mapping NVD's CPE strings back to package names. The live registries and GitHub API are used only for the daily refresh. Every record is validated against a Pydantic schema before being written to TimescaleDB, whose hypertables suit the one-row-per-package-per-week structure this data takes. Advisories for the package universe are separately chunked and embedded into Qdrant for retrieval.

Component 2 — Risk model service

This is the predictive core, and it is now a classification problem rather than a forecasting one. Each row is one package at one weekly snapshot, labeled positive if an advisory event falls between seven and ninety days ahead. The model is LightGBM over seven feature families, benchmarked at every step against three baselines, calibrated with isotonic regression, and logged to MLflow. Promotion from Staging to Production requires beating the current production model's PR-AUC on the latest closed validation window by more than the bootstrap confidence interval — a deliberate change from version 1's "auto-promote when F1 improves", which would promote on noise.

The Temporal Fusion Transformer was dropped because it is the wrong tool. This is binary classification on windowed tabular features with few positives, not multi-horizon sequence forecasting. LightGBM trains in minutes and will most likely win, and "why TFT?" is a question that would not have had a good answer in an interview. It survives only as a stretch goal, permitted in if it beats LightGBM on the same split.

Component 3 — Takeover detectors

Maintainer takeovers are rare, behaviourally distinct, and far too few to train a model on — there are perhaps a dozen well-documented incidents across npm and PyPI. Treating them as a supervised learning problem would be dishonest, so they are handled with six transparent rule-based detectors instead: a first-time publisher on a version, a registry–repository mismatch, a new install hook appearing in a patch release, a surprise young dependency, a dormancy break by a new account, and a maintainer handover. Each emits a flag with a human-readable explanation, and the combined score is a weighted sum whose weights are fixed on the design set only.

The evaluation protocol matters more than the detectors. Thresholds are tuned on three design incidents (event-stream, ua-parser-js, ctx), then frozen. The held-out set — coa and rc, colors and faker, node-ipc, ultralytics, @solana/web3.js — is touched exactly once. A control set of 500 random top packages over the same period gives a false-alarm rate per 1,000 package-months, because a detector that flags everything catches everything.

Component 4 — Security assistant

The conversational layer, and the only real agent in the system. It has four tools — risk score lookup, takeover flags, manifest scanning, and advisory search — and performs hybrid retrieval from Qdrant over the advisories for the 10,000-package universe rather than all 200,000+ CVE records. A smaller, scoped corpus means faster evaluation cycles and the same demonstrated skill. Because security claims carry real consequences if wrong, a post-check rejects any answer that cites an advisory identifier which was not actually returned by a tool call, which is a stronger guarantee than asking the model to be careful.

Component 5 — Evaluation service

The component that elevates this from a demo to an LLMOps system, and the one that is never cut. It does two distinct jobs. First, it scores assistant responses using RAGAS across faithfulness, answer relevance and context precision, against a curated 60-query golden dataset. Second, it independently backtests the risk model week over week as new advisories are published and past label windows close. If faithfulness drops, or forecasting precision decays, it raises a Prometheus alert and can trigger a retraining run.

3. Dataset and evaluation design

This is the part of the project that decides whether any of the rest is worth anything, and it is the part to build slowly. It is also the strongest material for an interview, because very few candidates can discuss leakage and confounding from their own work.

Labels

One row is one package at one weekly snapshot date t , over a universe of the top 5,000 PyPI and top 5,000 npm packages by downloads that have a mapped GitHub repository. For each OSV advisory, the event date is the earlier of the advisory's published date and the release date of the first fixed version. A snapshot is positive if an event date falls in the window from seven to ninety days after t . Snapshots with an event inside the first seven days are dropped rather than labeled negative, and that blackout window is the single most important line of code in the project.

The reason is leakage. Fix commits and security releases routinely land before the advisory is published. A model anchored on the publication date can quietly learn to detect a fix already in progress, which looks like excellent prediction on paper and is worthless in production. Version 1 would almost certainly have produced impressive, meaningless numbers this way.

Leakage controls

- Every feature is computed only from data dated before t , enforced by a unit test that fails if any feature query reaches past it.

- Features that directly encode security work — commits or issues mentioning CVE, security or vulnerability — are excluded, with one deliberate ablation including them to quantify the size of the leakage effect.
- Splits are strictly chronological: train on label windows closing before January 2024, validate on the first half of 2024, test on the second half of 2024 and 2025. Never shuffled at random.
- A package-grouped split is reported alongside the time split, to show the model is not simply memorising repeat offenders.

The model ladder and the metrics

Nothing is claimed as a result until it clears the rung below it. B0 ranks by downloads, B1 by prior advisory count, B2 is logistic regression on both; M1 is LightGBM on all seven feature families and M2 adds calibration. Every rung is logged to MLflow, so the comparison is a recorded fact rather than a memory.

PR-AUC is the headline metric, because ROC-AUC looks flattering at a low base rate. Precision@k and recall@k at k = 50, 100 and 500 per week match the real use of the system: which packages should a team review first? Lift over B2 at the same k is the number that answers the thesis. Everything is reported with bootstrap confidence intervals, accompanied by SHAP attribution, per-family ablations, and hand-read error analysis on twenty false positives and twenty false negatives.

What a negative result would mean

It is entirely possible that repository-health signals add nothing beyond popularity and advisory history. If that happens, the project is not a failure — it is a finding, and it should be reported as one, with the ablations that show which families were checked and how thoroughly. A well-run negative result with clean methodology is a stronger interview story than a suspiciously clean positive one, because it demonstrates exactly the judgement that separates an engineer who ships models from one who ships confident nonsense.

4. Why the evaluation layer is the differentiator

Most portfolio AI projects stop at "I built a RAG chatbot". Augur is designed so that the chatbot is the least interesting part of the system from an engineering standpoint, and the evaluation and monitoring infrastructure is the most interesting part. Concretely: a GitHub Actions workflow runs the full RAGAS evaluation suite against the golden dataset on every commit that touches a prompt template, and the build fails if faithfulness regresses past a defined threshold — the same discipline applied to model code that most teams only apply to application code.

The risk model gets the same treatment. Each week, newly closed label windows produce fresh ground truth, the current production model is scored against it, and the result is written to MLflow and exposed as a Prometheus metric. A Grafana dashboard tracks retrieval quality and ranking precision on the same screen as request latency and container health, because in a real AI system "is the model still right" is as operationally important as "is the server still up". This is also what makes the registry gate meaningful: promotion is tied to a significant improvement on a window the candidate model has never seen.

5. Twelve-week roadmap

Month 1 — Leakage-safe dataset and the first honest number

Week 1 — Labels and universe. Parse the OSV bulk export for PyPI and npm. Map packages to GitHub repositories through deps.dev and fix the universe at the top 10,000 packages by downloads. Measure and write down the mapping coverage you actually achieve, because it constrains everything downstream.

Week 2 — Historical extraction. Pull GH Archive activity, Scorecard history and download history out of BigQuery and into TimescaleDB, filtering by date partition and repository list and materialising small intermediate tables to stay inside the free tier. Done when one package's full history is queryable end to end.

Week 3 — The dataset itself. Build the weekly snapshot table, the label definition with its seven-day blackout, the chronological splits, and the leakage unit test. Record the base rate and the class counts. This is the most important week in the project, and the one most likely to take longer than planned.

Week 4 — Baselines and first model. Baselines B0 through B2, then the first LightGBM, all logged to MLflow. FastAPI shell and the Docker Compose stack. **Checkpoint 1:** if repository-mapping coverage is below roughly 60% of the universe, shrink to a single ecosystem — npm has the better per-version publisher data. If LightGBM does not beat B2 at all, keep going; the thesis becomes the negative result and the next three weeks become the explanation of why.

Month 2 — Explanation, detectors, and the evaluation loop

Week 5 — Understanding the model. Feature-family ablations, SHAP attribution, isotonic calibration and hand-read error analysis on false positives and false negatives. Write the findings into the repository rather than merely plotting them.

Week 6 — Takeover detectors. Build the six detectors against the design set, rebuilding each incident timeline from primary sources — advisories and maintainer post-mortems — and verifying every date before it goes anywhere near the README.

Week 7 — The held-out test. Freeze the thresholds, then run the held-out incident set exactly once, with the 500-package control set for false alarms. Record the results as they come out, including the ones you do not like.

Week 8 — Evaluation service. The weekly backtest job on newly closed windows, the registry promotion gate, custom Prometheus metrics and the Grafana dashboard. **Checkpoint 2:** if you are more than a week behind, apply the first two cuts before starting the assistant. The evaluation loop ships before the chatbot, always.

Month 3 — Assistant, CI gates, and deployment

Week 9 — Security assistant. Four tools, scoped retrieval, the citation post-check, and the custom orchestrator: a shared state object, an intent classifier that routes queries, a retry decorator with exponential backoff, and a fallback chain that degrades gracefully to pure retrieval if the model call fails.

Week 10 — Golden set and CI gate. Curate 60 queries spanning factual lookups, risk assessment, comparative questions and hallucination-resistance edge cases. Wire RAGAS into GitHub Actions. Done when a deliberately regressed prompt fails the build in front of you.

Week 11 — Packaging. A standalone GitHub Action that scans a `requirements.txt` or `package.json` and returns a ranked risk report inline, running on a public demo repository. This turns the project from something you demo into something other engineers can install. The React dashboard is optional and comes last.

Week 12 — Documentation and demo. Helm chart and Oracle OKE deployment, or Docker Compose if that was cut. Architecture diagram, demo video, and a README carrying the real test-set numbers with their confidence intervals. **Checkpoint 3:** no resume bullet leaves the building without its number traceable to the README.

6. Risk management — what to cut if time runs short

If the twelve weeks compress under coursework pressure, cut in this order. Drop the React dashboard first — a Grafana dashboard plus the GitHub Action output carries the same signal for a fraction of the effort. Drop Airflow second and replace it with a cron-scheduled script, documenting the DAG design in the README; the orchestration concept is demonstrated either way. Drop the Kubernetes migration third and stay on Docker Compose with a Helm chart that is written and linted but not deployed — a real loss against the skill gap, but the least damaging to the core narrative. Reduce the golden evaluation dataset from 60 queries to 30 before cutting it entirely.

Never cut the baselines, the leakage test, the chronological splits, the registry gate, or the CI eval gate. Version 1's cut list protected the RAGAS integration, which was right as far as it went, but the scientific controls matter more. Without them Augur is a student project with a confident README; with them it is defensible work, whatever the numbers turn out to be.

There is one more risk worth naming explicitly: scope. Version 1 asked for seven new technologies in twelve weeks, solo, alongside final-year coursework, interview preparation and applications. The likely failure mode was broad and shallow, and interviewers probe depth. Version 2 front-loads the data and the model, pushes Kubernetes and Airflow to the end where they are cheap to drop, and attaches a kill criterion to each month so the decision to narrow scope is made on evidence rather than on the Sunday night before a deadline.

7. Portfolio and interview positioning

Augur is designed to answer the three questions a strong technical interviewer will ask about any AI project, in order. **How do you know it works?** Because the ranking was measured against a popularity baseline on a chronologically held-out test set, with confidence intervals, and the lift over that baseline is reported whatever it turned out to be. **What happens when it's wrong?** The evaluation service continuously scores both retrieval quality and ranking precision, with automated alerts and CI gates that block regressions before they reach production. **Why should I believe this generalises?** Because the takeover detectors were frozen before the held-out incidents were touched, and the false-alarm rate on a clean control set is published alongside the detection results.

When discussing this project, lead with the baseline question — "my first question was whether health signals beat popularity at all" — then the leakage story, then the evaluation loop, and only then the architecture. Those first three are what make the conversation memorable, and they are the parts of the project hardest for other candidates to replicate convincingly. If XZ Utils comes up, say that it inspired the maintainer-trust features but sits outside the scope of a PyPI and npm system, and pivot to the event-stream result. That answer demonstrates more security judgement than the original claim ever would have.

Finally, on the resume itself: write the bullets as templates during the build and fill the brackets from the README afterwards. A modest real result with honest analysis interviews better than a suspiciously clean one, and an interviewer who probes an invented figure ends the conversation there.